

Marel Norm Tracking App — Backend

Kompletan Flow: Work Logovi → Daily → Monthly Report

Datum: 10. juni 2026.

Pregled arhitekture

Sistem koristi **asinhroni recalkulacioni mehanizam** zasnovan na bazi podataka (PostgreSQL). Umesto direktnog azuriranja reporta pri svakom unosu, promene se stavljaju u red cekanja i obradjuju pozadinskim workerima. Ovo obezbedjuje visoku propusnost i konzistentnost podataka cak i pri konkurentnim zahtevima.

1. Ulazna tacka — Upisivanje work logova

Fajl: `work_log/WorkLogController.java`

Jedini REST endpoint za kreiranje, izmenu i brisanje work logova:

```
POST /api/work-logs/batch
Body: CreateUpdateWorkLogsRequest { create: [...], update: [...], deleted: [...] }
```

Controller je tanak — samo delegira na **WorkLogService.handleBatch()**.

2. WorkLogService.handleBatch() — Mutacija + Enqueue

Fajl: `work_log/WorkLogService.java`

Metoda je anotirana sa **@Transactional**. Unutar jedne transakcije:

- **Create:** `workLogMapper.toEntity()` + `repository.saveAll()`
- **Update:** fetch po IDs, `workLogMapper.updateEntity()`, `save`
- **Delete (soft):** `existing.setIsActive(false)`, `save` — nema fizickog brisanja

Nakon mutacija, za svaki zahvaceni **WorkShift** (deduplicirano po shift ID-u):

```
recalcQueueService.enqueueDailyJob(shift, "WORK_LOG_MUTATION")
// UPSERT u daily_report_recalc_queue:
// - Ako job ne postoji: INSERT sa status='PENDING'
// - Ako vec postoji: UPDATE version++ i reset na 'PENDING'
// - Ako je IN_PROGRESS: ostavlja status, samo bumpa version
```

Na kraju transakcije publishuje se Spring Application Event:

```
eventPublisher.publishEvent(new DailyRecalcRequestedEvent(Type.DAILY))
```

Ovaj event se isporucuje POSLE commita (AFTER_COMMIT), kako bi workeri videli upisane jobove.

3. Wake Signal — Budjenje workera

Fajlovi: `DailyRecalcRequestedEventListener.java`, `RecalcWorkerWakeSignal.java`

Listener osluskuje **DailyRecalcRequestedEvent** i poziva **wakeSignal.signalAll()**, koji radi **Object.notifyAll()** na deljenom monitoru. Worker threadovi koji su u **monitor.wait(timeoutMs)** (idle stanje) se odmah bude i krecu

sa procesiranjem.

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onRecalcRequested(DailyRecalcRequestedEvent event) {
    wakeSignal.signalAll();
}
```

4. DbQueueWorkerManager — Pozadinski threadovi

Fajl: `report_worker/DbQueueWorkerManager.java`

Startuje se na `@EventListener(ApplicationReadyEvent.class)` — cim se aplikacija podigne. Kreira konfigurisani broj Java platform threadova:

- N x **dailyLoop** threadova ("daily-worker-1", "daily-worker-2", ...)
- M x **monthlyLoop** threadova ("monthly-worker-1", ...)

Logika jednog **dailyLoop**:

```
while (running) {
    // Svakih 20 iteracija: oporavi stuck IN_PROGRESS jobove
    if (loop++ % 20 == 0) requeueStuckDailyJobs();

    int processed = dailyReportWorker.processBatch(batchSize, workerId, loopBudget);

    if (processed == 0) {
        waitForWakeOrTimeout(idleMs); // ceka na signal ili timeout
        idleMs = min(maxIdleMs, idleMs * 2); // exponential backoff idle
    } else {
        idleMs = minIdleMs; // odmah ponovi ako ima posla
    }
}
```

Isti pattern za **monthlyLoop**. Graceful shutdown: `@PreDestroy → running=false + thread.interrupt()`.

5. Claim + Pessimistic Lock — RecalcQueueService

Fajl: `recalc_queue/RecalcQueueService.java`

Metoda **claimDailyJobIds(batchSize, workerId)** koristi **FOR UPDATE SKIP LOCKED** kako bi u jednom SQL upitu i selektovala i zakljucala jobove, bez blokiranja izmedju workera:

```
WITH c AS (
    SELECT id FROM daily_report_recalc_queue
    WHERE status = 'PENDING' AND retry_count < maxRetry
    ORDER BY requested_at, id
    LIMIT batchSize
    FOR UPDATE SKIP LOCKED
)
UPDATE daily_report_recalc_queue q
SET status = 'IN_PROGRESS', claimed_at = NOW(), claimed_by = workerId
FROM c WHERE q.id = c.id
RETURNING q.id
```

Isti SQL pattern se koristi i za **monthly** jobove. Ovo garantuje da 2 workera nikada ne uzmu isti job.

6. DailyRecalcService — Obrada daily job-a

Fajl: `report_worker/DailyRecalcService.java`

Metoda **processJob(jobId)** je podeljena u 2 faze radi minimizovanja vremena drzanja locka:

Read faza (van transakcije):

- Provera statusa job-a — skip ako nije IN_PROGRESS
- Cita sve aktivne work logove za WorkShift (**findActiveLogsWithRefsForShift**)

Write faza (u TransactionTemplate):

- Fetch WorkShift, Employee, EmployeeRecord (getOrCreate za mesec)
- Fetch ili create DailyReport za ovaj shift
- **Stale guard:** re-lock queue job sa FOR UPDATE, proverava version — ako se promenio dok smo racunali → reschedule na PENDING
- **Bulk delete** starih DailyReportCategory-a: deleteAllByDailyReportId()
- **buildCategories()** — rebuild svih kategorija
- **fillDailyTotals()** — agregacija po tipu
- Save DailyReport + kategorije
- Queue job → DONE
- enqueueMonthlyJob() → UPSERT u monthly_report_recalc_queue
- WebSocket push: sendDailyReportUpdate() → /topic/reports/daily

buildCategories() — Logika racunanja:

Grupise logove po **WorkCodeCategory**, za svaku kategoriju racuna:

- totalMinutes, totalQuantity, totalScrap — suma svih logova u kategoriji
- **performanceRate** = $(\log.\text{hourlyOutput} / \text{operation.minNorm}) \times 100$ | 100 ako operacija nema normu
- **approvedPerformanceRate** = $\min(\text{performanceRate}, \text{maxEfficiencyPercent})$ — cap iz AppSettings
- Weighted average koeficijent po trajanju (minuti × stopa) / ukupni minuti
- **totalWeightedNormMinutes** = totalMinutes × approvedPerformanceCoefficient

fillDailyTotals() — Agregacija u DailyReport:

- totalShiftMinutes, totalWorkMinutes
- totalAbsencePaidMinutes, totalAbsenceUnpaidMinutes
- totalSickLeavePaidMinutes, totalSickLeaveUnpaidMinutes
- totalQuantity, totalScrap, totalWeightedNormMinutes
- performanceCoefficient, approvedPerformanceCoefficient, performanceRate
- **isMealAllowed** = true ako totalWorkMinutes >= 240
- calcVersion++ (koliko puta je recalkulisan)

7. MonthlyRecalcService — Obrada monthly job-a

Fajl: report_worker/MonthlyRecalcService.java

Isti dvofazni pattern kao DailyRecalcService:

Read faza (van transakcije):

- findByEmployee_IdAndWorkDateBetween(start, end) — svi DailyReport-i za mesec
- findAllByDailyReportIds() — sve DailyReportCategory za te reporte

Write faza (u TransactionTemplate):

- Fetch/create EmployeeRecord za mesec
- Fetch ili create MonthlyReport
- **Stale guard** — isti version check kao daily
- **Bulk delete** starih MonthlyReportCategory
- **buildMonthlyCategories()** — grupisu se DailyReportCategory po WorkCodeCategory, sumiraju minute/qty/scrap
- **fillMonthlyTotals()** — agregira sve DailyReport-e za mesec
- Save MonthlyReport + kategorije (bumpa @Version — Hibernate optimistic lock)
- Queue job → DONE
- WebSocket push: sendMonthlyReportUpdate() → /topic/reports/monthly

fillMonthlyTotals() — Agregacija u MonthlyReport:

- Sumira totalShiftMinutes, totalWorkMinutes, absence/sickLeave breakdown iz svih DailyReport-a
- mealAllowanceNum — broj dana sa isMealAllowed=true
- totalApprovedMinutes = $\Sigma(\text{weightedNormMinutes} \times \text{normMultiplier})$ po kategoriji, za period
- performanceCoefficient = totalWeightedNormMinutes / totalShiftMinutes
- calcVersion++

8. Retry, Failure i Stuck Recovery

Fajlovi: DailyRecalcService.markFailed(), MonthlyRecalcService.markFailed()

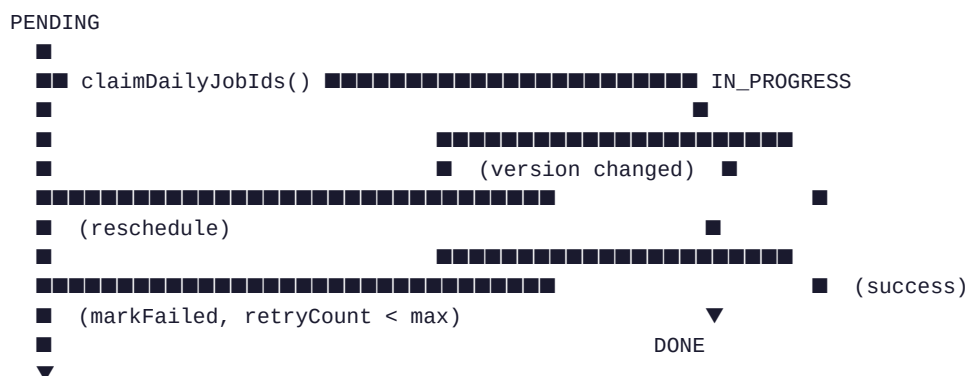
Ako **processJob()** baci izuzetak, worker poziva **markFailed(jobId, errMsg)**:

```
retryCount++
if (retryCount < maxRetry):
    status = 'PENDING'
    requestedAt = now + exponentialBackoff(retryCount) // base × 2^retry, max 5 min
else:
    status = 'FAILED' // permanentno neuspesan
```

Stuck Recovery — svakih 20 loop iteracija:

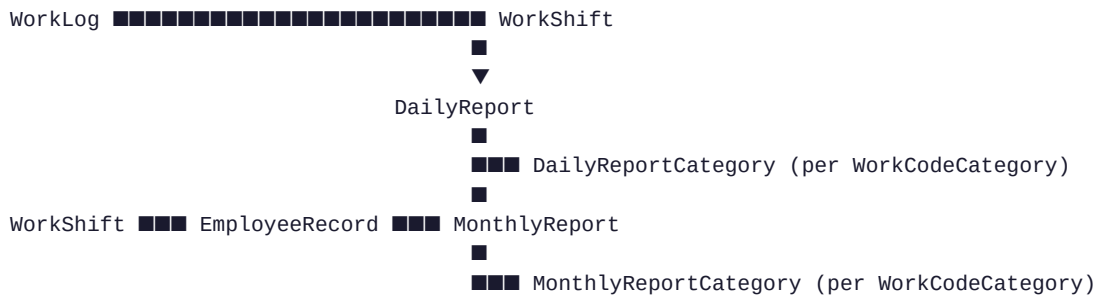
```
requeueStuckDailyJobs(stuckTimeoutSeconds, batchLimit)
// Pronalazi IN_PROGRESS jobove koji su stuck duze od timeout-a
// Resetuje ih na PENDING sa exponential backoff i incrementuje stuck_count
```

9. Status dijagram queue job-a



FAILED (retryCount >= maxRetry)

10. Ključni entiteti i relacije



Queue tabele:

daily_report_recalc_queue (per WorkShift, UNIQUE on work_shift_id)
monthly_report_recalc_queue (per employee + year + month, UNIQUE on (employee_id, year, month))

Sazetak (za agent)

STACK: Spring Boot 4 / Java 21 / PostgreSQL

ULAZNA TACKA:

POST /api/work-logs/batch
→ WorkLogController
→ WorkLogService.handleBatch() [Transactional]

CHAIN:

- handleBatch()
 - mutira work_log (create/update/soft-delete)
 - enqueueDailyJob() → UPSERT daily_report_recalc_queue (PENDING)
 - publishuje DailyRecalcRequestedEvent
- DailyRecalcRequestedEventListener [AFTER_COMMIT]
 - wakeSignal.signalAll() → budi worker threadove
- DbQueueWorkerManager [startuje se na ApplicationReadyEvent]
 - dailyLoop / monthlyLoop threadovi
 - while(running): processBatch() | waitForWakeOrTimeout()
- DailyReportWorker.processBatch()
 - RecalcQueueService.claimDailyJobIds() [FOR UPDATE SKIP LOCKED]
 - DailyRecalcService.processJob(jobId)
- DailyRecalcService.processJob()
 - read phase: fetchuje work logove
 - write TX:
 - bulk delete DailyReportCategory
 - rebuild kategorija (performance rate, weighted koeficijent)
 - fillDailyTotals() (WORK/ABSENCE/SICK_LEAVE agregacija)
 - save DailyReport + categories
 - job → DONE
 - enqueueMonthlyJob() → UPSERT monthly_report_recalc_queue (PENDING)
 - WebSocket push: /topic/reports/daily
- MonthlyReportWorker.processBatch()
 - RecalcQueueService.claimMonthlyJobIds() [FOR UPDATE SKIP LOCKED]
 - MonthlyRecalcService.processJob(jobId)
- MonthlyRecalcService.processJob()

```
read phase: fetchuje sve DailyReport-e + kategorije za mesec
write TX:
  bulk delete MonthlyReportCategory
  rebuild monthly kategorija (sum po WorkCodeCategory)
  fillMonthlyTotals() (sum daily reporta za mesec)
  save MonthlyReport + categories (bumpa @Version)
  job → DONE
  WebSocket push: /topic/reports/monthly
```

RETRY:

```
Exception → markFailed() → exponential backoff retry (do maxRetry) → FAILED
Stuck IN_PROGRESS recovery: requeueStuckDailyJobs/requeueStuckMonthlyJobs svakih 20 loopova
```

STATUS FLOW QUEUE JOBA:

```
PENDING → IN_PROGRESS → DONE
  ■ PENDING (reschedule - version mismatch ili retry)
  ■ FAILED (maxRetry exceeded)
```